

PER2-HALO: QC peptide-level quantification and filter

Tom Smith

2026-09-26

Contents

1 Purpose	1
2 Setup	3
3 Read the peptide data into a QFeatures object	3
3.1 Defining the contaminant proteins	3
3.2 Reading in the peptide-level quantification data	3
3.3 Extracting the experiment to sample mapping	4
3.4 Creating the QFeatures object	5
4 Filter the peptide-level quantification data	6
5 Normalise the peptide-level quantification data	7
5.1 Normalise	7
5.2 Normalise to histone proteins	8
6 Removing peptides with too many missing values	11
7 Inspecting the number of peptides and proteins through the processing steps	12
7.1 Samples per peptide	12
7.1.1 Samples per Protein	14
8 Save outputs	15
9 Session info	15

1 Purpose

This notebook reads the peptide-level output from the MaxQuant search of the PER2-HALO immunoprecipitation (IP) experiment (PER2-HALO fusion vs free HALO tag, 3 replicates each), applies routine peptide-level quality filtering, and normalises the abundances, producing a **QFeatures** object ready for protein-level summarisation and statistical testing in notebook 2.

The IP design means the two conditions are expected to differ substantially in their overall protein composition (a free HALO tag pulls down a different, and likely smaller, set of interactors than the PER2-HALO fusion), so particular attention is paid to the choice of normalisation strategy: standard median normalisation, which assumes the two conditions have a similar overall abundance distribution, is compared against normalisation to a set of proteins (histones) expected to be stable regardless of condition.

Input:

- `raw/HALO/modificationSpecificPeptides.txt.gz` — MaxQuant peptide-level (LFQ intensity) quantification.
- `raw/HALO/peptides.txt.gz` — used only for its `Sequence to Leading.razor.protein` mapping, to standardise protein assignment.
- `raw/HALO/proteinGroups.txt.gz` — protein-level search results, used here for the `Q.value` (FDR) column.
- `raw/HALO/mqpar.xml` — the MaxQuant parameter file, used to recover the mapping from MaxQuant's internal experiment labels to sample names.
- `external/uniprot2details_unique.rds` from notebook 0 — UniProt protein annotations.

Output:

- `notebooks/output/HALO_lfq_qf.rds` — a `QFeatures` object retaining every processing stage (raw, filtered, normalised, histone-normalised, and missing-value-filtered peptide assays), consumed by notebook 2.

Analysis:

1. **Read data** — read the peptide-level quantification, standardise the `Leading.razor.protein` assignment for peptides matching the transgene sequences (see below), and join in the protein annotations and FDR (`Q.value`) from the protein-level search.
2. **Sample mapping** — recover the mapping from MaxQuant's internal experiment names to condition/replicate from the search parameter file, and build the `QFeatures` object.
3. **Zero handling** — replace MaxQuant's reported zero intensities with `NA`, since mass spectrometry cannot assert a true zero abundance.
4. **Filtering** — remove contaminant peptides and peptides belonging to proteins that do not pass the 1% FDR threshold.
5. **Normalisation** — log2-transform, then compare standard median normalisation against normalisation to a set of stably-expressed histone proteins, and carry forward the histone-normalised assay as the one used downstream.
6. **Missing value inspection** — visualise missingness patterns and construct a peptide set filtered to a maximum of 4/6 missing values, primarily as a diagnostic of how much data a naive missing-value filter would discard.
7. **QC summary** — plot the number of peptides and proteins retained at each processing stage.
8. **Export** — save the full `QFeatures` object.

How to read the results: the assay actually carried forward into notebook 2 is `peptides_norm_hist` (histone-normalised, but *not* filtered for missing values). Notebook 2 summarises to protein level using `limpa`, which models missingness explicitly via a detection probability curve rather than requiring complete peptides, so there is no need to discard peptides with missing values before that step; the 4/6-missing-value-filtered assay (`peptides_filtered_missing`) constructed here is used only for the peptide/protein-count QC plots at the end of this notebook, not as an input to notebook 2.

2 Setup

Load required packages

```
library(biomasslmb)
library(ggplot2)
library(tidyr)
library(dplyr)
library(here)
library(QFeatures)
```

```
## Warning: package 'MultiAssayExperiment' was built under R version 4.5.3
```

```
## Warning: package 'S4Vectors' was built under R version 4.5.3
```

```
library(xml2)
```

Read in the protein details

```
uniprot2details <- readRDS(here('external/uniprot2details_unique.rds'))
```

3 Read the peptide data into a QFeatures object

We now start reading in the peptide-level quantification data from MaxQuant into a QFeatures object.

3.1 Defining the contaminant proteins

We need to remove contaminant proteins. Below, we parse the MaxQuant contaminants fasta to extract the IDs for the proteins.

```
contaminant_proteins <- get_maxquant_cont_accessions()
```

```
## No contaminants fasta supplied, defaulting to contaminants.fasta.gz from biomasslmb
```

3.2 Reading in the peptide-level quantification data

We need to read in both the proteinGroups.txt files from MaxQuant to identify the proteins which pass the 1% FDR threshold, and the modificationSpecificPeptides.txt file to get the peptide-level quantification.

The search database for this experiment includes separate entries for the free HALO tag construct (PER2HALO/PER2MNG) and, for peptides shared between the PER2-HALO fusion and endogenous PER2, MaxQuant does not always assign the UniProt accession (015055) as the leading razor protein. Below we override `Leading.razor.protein` for these two specific cases, so that every peptide from the tag construct is consistently grouped under a single pseudo-protein (HALO_TAG) and every peptide from the fusion/endogenous protein is grouped under the PER2 UniProt accession, regardless of what MaxQuant happened to assign.

```

pg_inf <- here('raw/HALO/proteinGroups.txt.gz')
pg_df <- read.delim(pg_inf)

seq2razor <- read.delim(here('raw/HALO/peptides.txt.gz')) %>%
  select(Sequence, Leading.razor.protein)

pep_inf <- here('raw/HALO/modificationSpecificPeptides.txt.gz')
pep_df <- read.delim(pep_inf)

# Merge with the leading razor protein column
pep_df <- merge(pep_df, seq2razor, by='Sequence')

per2_lines <- which(grepl('015055', pep_df$Proteins))
halo_lines <- which(pep_df$Proteins=='PER2HALO;PER2MNG')

pep_df$Leading.razor.protein[halo_lines] <- 'HALO_TAG'
pep_df$Leading.razor.protein[per2_lines] <- '015055'

# Merge with the protein details and Q-values
pep_df <- merge(pep_df, uniprot2details,
  by.x='Leading.razor.protein', by.y='UniprotID', all.x=TRUE) %>%
  merge(pg_df[,c('Majority.protein.IDs', 'Q.value')], 'Leading.razor.protein',
    by.y='Majority.protein.IDs', all.x=TRUE)

```

3.3 Extracting the experiment to sample mapping

MaxQuant refers to each raw file by an internal single-letter “experiment” label rather than a readable sample name. We recover the mapping between the two from the search parameter file (`mqpar.xml`), which lists the raw file paths (from which we parse the intended sample name) alongside the experiment labels in the same order. From the sample name we derive condition (HA = HALO-only control, P2H = PER2-HALO) and replicate.

```

mqpar_inf <- here('raw/HALO/mqpar.xml')
mqpar <- xml2::read_xml(mqpar_inf)

# Extract the file paths and experiment names
filepaths <- strsplit(
  xml_text(xml_find_all(mqpar, "`.`filePaths")), split='F:\\\\')[[1]]
filepaths <- filepaths[filepaths!='']

# Extract sample names from file paths
sample <- gsub(
  '(2565608863_Andrei_PER2_121225\\\\\\\\2565608863_Andrei_|.raw)', '', filepaths)

experiments <- strsplit(
  xml_text(xml_find_all(mqpar, "`.`experiments")), split='')[[1]]

# Checking that the lengths match
length(sample)==length(experiments)

```

```
## [1] TRUE
```

```

# Create the coldata dataframe
coldata <- data.frame('sample'=sample, 'experiments'=experiments) %>%
  separate(sample, into=c('sample', NA)) %>%
  distinct() %>%
  mutate(sample=sub('(HA|P2H)', '\\1_', sample)) %>%
  separate(sample, into=c('condition', 'replicate'), sep='_', remove=FALSE) %>%
  mutate(quantCols=sample) %>%
  tibble::column_to_rownames('sample')

# named list to map from experimental name to sample name
exp2sample <- coldata$quantCols
names(exp2sample) <- coldata$experiments

coldata %>%
  knitr::kable()

```

	condition	replicate	experiments	quantCols
HA_1	HA	1	A	HA_1
HA_2	HA	2	B	HA_2
HA_3	HA	3	C	HA_3
P2H_1	P2H	1	D	P2H_1
P2H_2	P2H	2	F	P2H_2
P2H_3	P2H	3	G	P2H_3

3.4 Creating the QFeatures object

Now we can finally create the QFeatures object. The abundance columns (32-37 in the merged data frame) correspond to the per-experiment **Intensity** columns MaxQuant reports; we rename them from the internal experiment label to the readable sample name using the `exp2sample` mapping derived above, and reorder the sample metadata (`coldata`) to match.

```

# Define the columns corresponding to the abundance values
abundance_cols <- 32:37

# Rename the abundance columns to the sample names
colnames(pep_df)[abundance_cols] <- gsub(
  'Intensity.', '', colnames(pep_df)[abundance_cols])

colnames(pep_df)[abundance_cols] <- sapply(
  colnames(pep_df)[abundance_cols], function(x) exp2sample[[x]])

# Reorder the coldata to match the order of the abundance columns
coldata <- coldata[colnames(pep_df)[abundance_cols],]

# Create the QFeatures object
lfq_qf <- readQFeatures(assayData = pep_df,
  colData = coldata,
  quantCols = abundance_cols,
  name = 'peptides_raw')

```

```
## Checking arguments.
```

```
## Loading data as a 'SummarizedExperiment' object.
```

```
## Formatting sample annotations (colData).
```

```
## Formatting data as a 'QFeatures' object.
```

```
# Set the colData for the peptides_raw experiment  
colData(lfq_qf[['peptides_raw']]) <- colData(lfq_qf)
```

MaxQuant has reported 107855 quantification values as zero. These should be replaced with NA, since mass spectrometry is not capable of asserting that the protein had zero abundance in the sample.

```
lfq_qf[['peptides_raw']] <- QFeatures::zeroIsNA(lfq_qf[['peptides_raw']])
```

4 Filter the peptide-level quantification data

We will now filter the peptide-level quantification data to remove contaminants and proteins which do not pass the 1% FDR threshold.

```
lfq_qf[['peptides_filtered']] <- biomasslmb::filter_features_mq_dda(  
  lfq_qf[['peptides_raw']],  
  contaminant_proteins=contaminant_proteins,  
  filter_contaminant=TRUE,  
  filter_associated_contaminant=TRUE,  
  unique_master=FALSE)
```

```
## Filtering data...
```

```
## 35885 features found from 4088 master proteins => Input
```

```
## 35816 features found from 4046 master proteins => Removed hits to decoy database
```

```
## 246 contaminant proteins supplied
```

```
## 926 proteins identified as 'contaminant associated'
```

```
## 35093 features found from 4003 master proteins => contaminant features removed
```

```
## 34816 features found from 3979 master proteins => associated contaminant features removed
```

```
## 34816 features found from 3979 master proteins => MaxQuant-labelled 'Contaminants' removed
```

```
## 34816 features found from 3979 master proteins => features without a master protein removed
```

```
## 33326 features found from 3961 master proteins => features without quantification removed
```

```
lfq_qf <- filterFeatures(lfq_qf, i='peptides_filtered', ~ Q.value <= 0.01)
```

```
## 'Q.value' found in 2 out of 2 assay(s).
```

```
biomasslmb:::message_parse(rowData(lfq_qf[['peptides_filtered']]),
                           "Leading.razor.protein",
                           "Proteins with Q-Value > 0.01 removed")
```

```
## 33324 features found from 3959 master proteins => Proteins with Q-Value > 0.01 removed
```

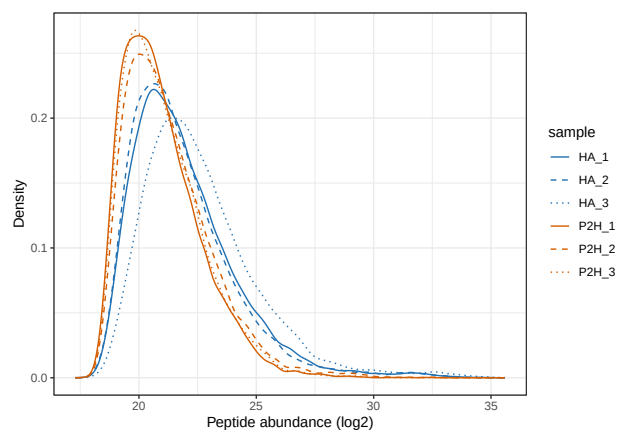
5 Normalise the peptide-level quantification data

For the normalisation and summarisation, we need our quantification values to be on a log-scale, so we log-transform below

```
lfq_qf[['peptides_filtered']] <- QFeatures::logTransform(
  lfq_qf[['peptides_filtered']], base=2)
```

Next, we plot the peptide intensities.

```
# Plot the peptide-level quantification distributions per sample
plot_quant(lfq_qf[['peptides_filtered']], log2transform=FALSE, method='density') +
  scale_colour_manual(values=rep(get_cat_palette(4), each=3)) +
  aes(linetype=sample) +
  scale_linetype_manual(values=rep(1:3, times=4)) +
  xlab('Peptide abundance (log2)')
```



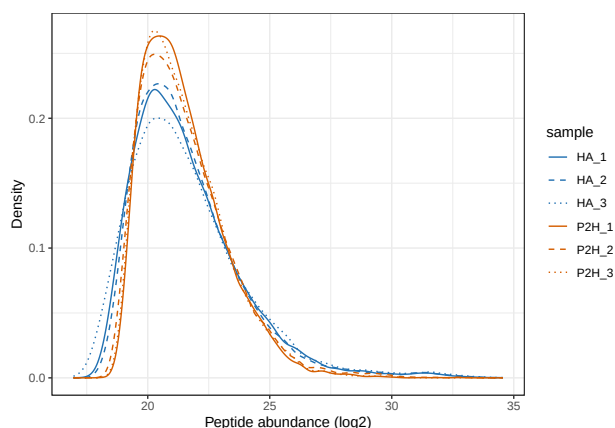
5.1 Normalise

Since the same amount of sample was injected for each sample, we could normalise to remove differences in the overall distribution of abundances. This is performed below using median normalisation.

```
lfq_qf[['peptides_norm']] <- QFeatures::normalize(lfq_qf[['peptides_filtered']],
                                                  method='diff.median')
```

Below, we can check this normalisation step has worked as intended.

```
# Plot the peptide-level quantification distributions per sample
plot_quant(lfq_qf[['peptides_norm']], log2transform=FALSE, method='density') +
  scale_colour_manual(values=rep(get_cat_palette(4), each=3)) +
  aes(linetype=sample) +
  scale_linetype_manual(values=rep(1:3, times=4)) +
  xlab('Peptide abundance (log2)')
```



5.2 Normalise to histone proteins

Median normalisation is suitable when we expect the overall distribution of protein abundances to be similar between the conditions. However, we don't expect the overall distribution of protein abundances to necessarily be similar between the conditions here, since one is a free HALO tag and the other is PER2-HALO. We should also consider normalising to a set of proteins which we expect to be constant across the samples. Histone proteins are often used for this purpose since they are generally stable in abundance. Below, we identify the histone proteins in our dataset and explore their abundance post normalisation. They are more abundant in the HALO condition, suggesting that global median normalisation may be more inappropriate.

```
hist_df <- uniprot2details %>%
  filter(grepl('^Histone H', Protein.names))

hist_df %>%
  select('UniprotID'=UniprotID, 'Gene Name'=Gene.Names.First, 'Protein names'=Protein.names) %>%
  knitr::kable()
```

Gene	
UniprotName	Protein names
P07305H1-0	Histone H1.0 (Histone H1') (Histone H1(0)) [Cleaved into: Histone H1.0, N-terminally processed]
P10412H1-4	Histone H1.4 (Histone H1b) (Histone H1s-4)
P16104H2AX	Histone H2AX (H2a/x) (Histone H2A.X)
P16403H1-2	Histone H1.2 (Histone H1c) (Histone H1d) (Histone H1s-1)
P49450CENPA	Histone H3-like centromeric protein A (Centromere autoantigen A) (Centromere protein A) (CENP-A)
P58876H2BC5	Histone H2B type 1-D (H2B-clustered histone 5) (HIRA-interacting protein 2) (Histone H2B.1 B) (Histone H2B.b) (H2B/b)

Gene	Protein names
UniprotName	
P62805H4C1	Histone H4
P62807H2BC4	Histone H2B type 1-C/E/F/G/I (Histone H2B.1 A) (Histone H2B.a) (H2B/a) (Histone H2B.g) (H2B/g) (Histone H2B.h) (H2B/h) (Histone H2B.k) (H2B/k) (Histone H2B.l) (H2B/l)
Q02539H1-1	Histone H1.1 (Histone H1a)
Q16778H2BC21	Histone H2B type 2-E (H2B-clustered histone 21) (Histone H2B-GL105) (Histone H2B.q) (H2B/q)
Q5TEC63-7	Histone H3-7
Q71DI3H3C15	Histone H3.2 (H3-clustered histone 13) (H3-clustered histone 14) (H3-clustered histone 15) (Histone H3/m) (Histone H3/o)
Q71UI9H2AZ2	Histone H2A.V (H2A.F/Z) (H2A.Z variant histone 2)
Q8IUE6H2AC21	Histone H2A type 2-B (H2A-clustered histone 21)
Q8N25H2BC26	Histone H2B type 3-B (H2B type 12) (H2B-clustered histone 26) (H2B.U histone 1)
Q92522H1-10	Histone H1.10 (Histone H1x)
Q93077H2AC6	Histone H2A type 1-C (H2A-clustered histone 6) (Histone H2A/l)
Q96A08H2BC1	Histone H2B type 1-A (Histone H2B, testis) (TSH2B.1) (hTSH2B) (Testis-specific histone H2B)
Q99878H2AC14	Histone H2A type 1-J (Histone H2A/e)
Q99879H2BC14	Histone H2B type 1-M (Histone H2B.e) (H2B/e)
Q99880H2BC13	Histone H2B type 1-L (Histone H2B.c) (H2B/c)

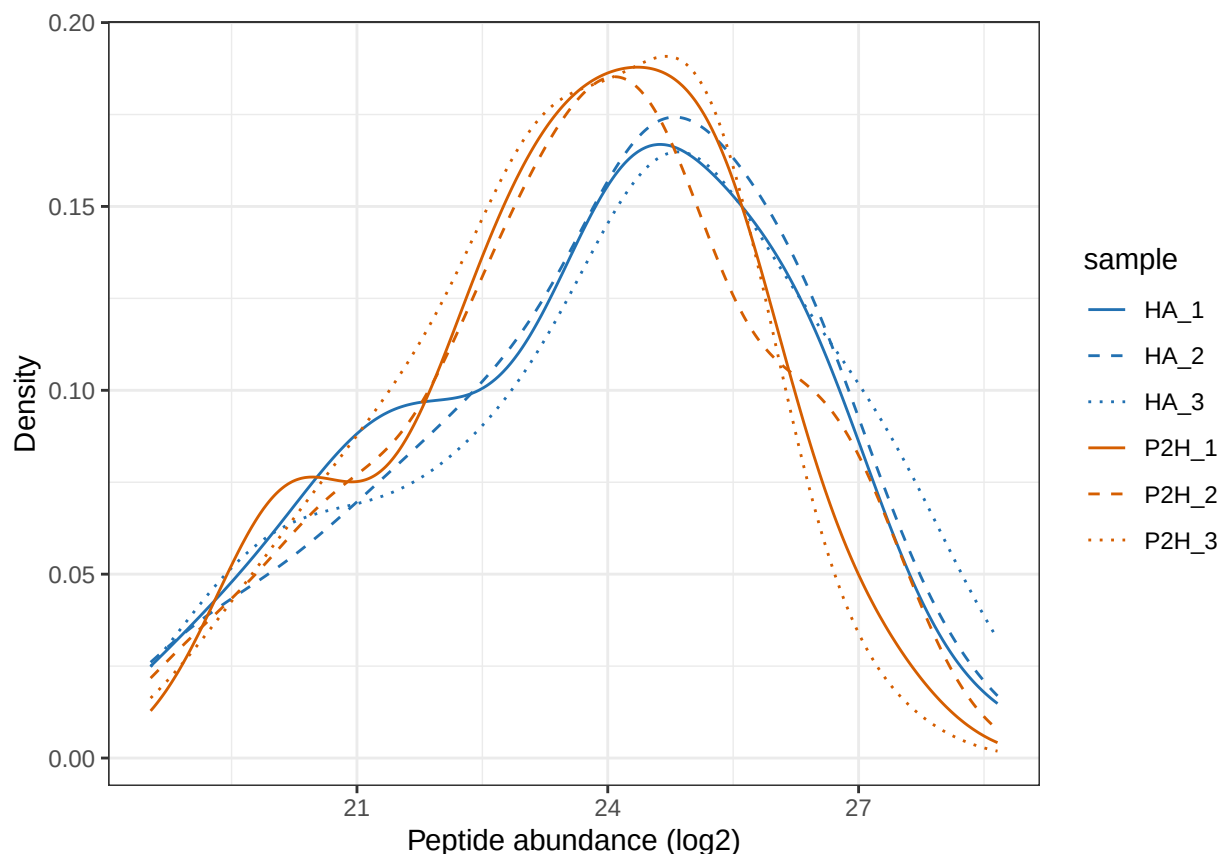
```

hist <- hist_df %>%
  pull(UniprotID)

# Plot the peptide-level quantification distributions per sample
plot_quant(lfq_qf[['peptides_norm']][
  rowData(lfq_qf[['peptides_norm']])$Leading.razor.protein %in% hist,],
  log2transform=FALSE, method='density') +
  scale_colour_manual(values=rep(get_cat_palette(4), each=3)) +
  aes(linetype=sample) +
  scale_linetype_manual(values=rep(1:3, times=4)) +
  xlab('Peptide abundance (log2)')

## Warning: Removed 86 rows containing non-finite outside the scale range
## ('stat_density()').

```



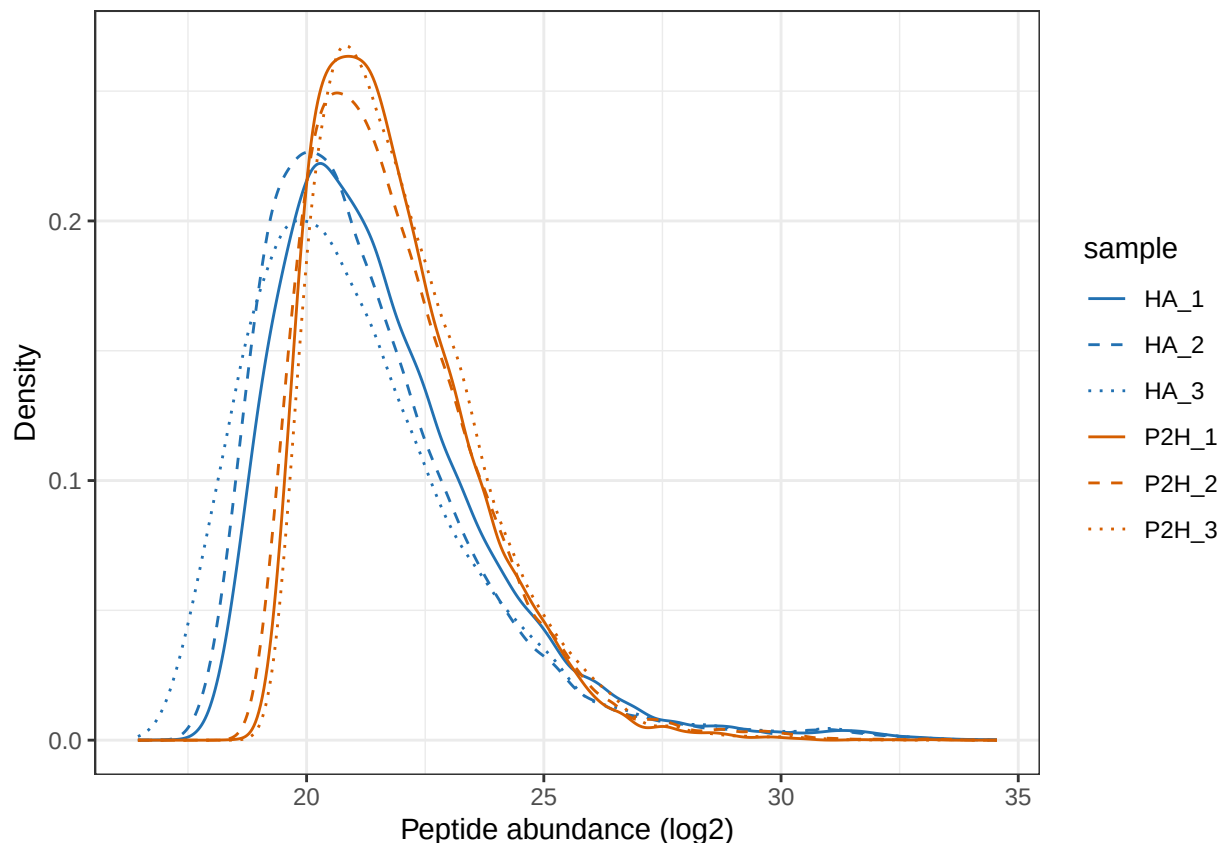
Below, we normalise to the histone proteins and replot the peptide-level abundance distributions. This is the normalised peptide-level assay (`peptides_norm_hist`) that is carried forward into notebook 2 for protein-level summarisation.

```
hist_medians <- get_medians(lfq_qf[['peptides_filtered']][
  rowData(lfq_qf[['peptides_filtered']])$Leading.razor.protein %in% hist,])

lfq_qf[['peptides_norm_hist']] <- center_normalise_to_ref(
  lfq_qf[['peptides_filtered']], medians = hist_medians, on_log_scale = TRUE)

# Plot the peptide-level quantification distributions per sample
plot_quant(lfq_qf[['peptides_norm_hist']], log2transform=FALSE, method='density') +
  scale_colour_manual(values=rep(get_cat_palette(4), each=3)) +
  aes(linetype=sample) +
  scale_linetype_manual(values=rep(1:3, times=4)) +
  xlab('Peptide abundance (log2)')
```

```
## Warning: Removed 96622 rows containing non-finite outside the scale range
## ('stat_density()').
```



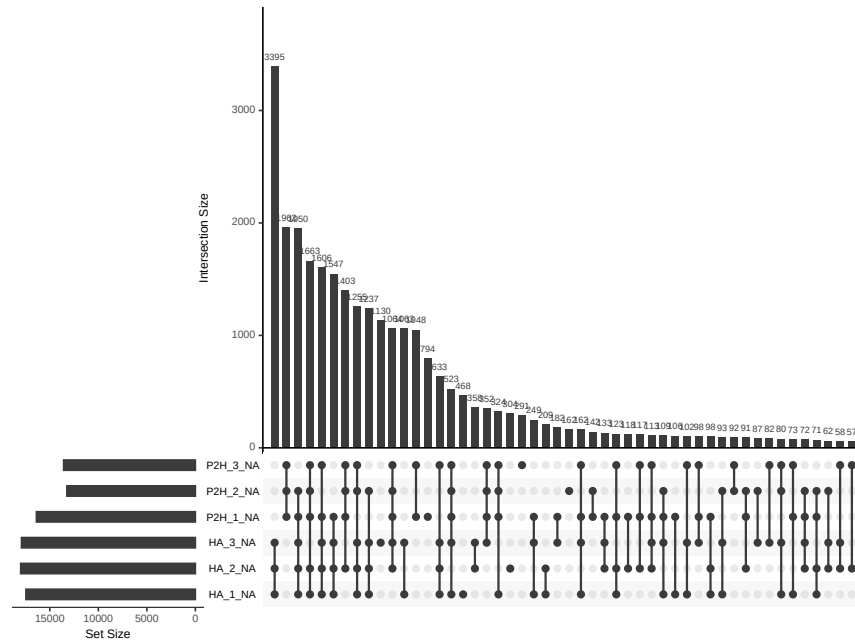
6 Removing peptides with too many missing values

For the estimation of protein-level abundances in the next notebook, we will use `limpa`. We do not need to exclude peptides with any missing values for this, since `limpa` explicitly models missingness via a detection probability curve rather than requiring complete data. Below we nonetheless inspect how much of the peptide set would be lost under a conventional missing-value filter, as a diagnostic; the resulting filtered assay is used only for the peptide/protein-count QC plots at the end of this notebook and is not itself passed on to notebook 2.

Below, we inspect the most common patterns for the missing values.

The missingness is sometimes consigned to all samples from a condition(s). It looks like the missingness is a mixture of missing at random (MAR) and missing not at random (MNAR) and is likely partly due to falling below detection limits.

```
plot_missing_upset(lfq_qf, i='peptides_norm_hist')
```



Here, we'll set a limit of maximum 4/6 missing values per peptide.

```
table(nNA(lfq_qf[['peptides_norm_hist']])$nNArows$nNA)
```

```
##
##      0      1      2      3      4      5
## 5272 3149 3683 6834 6325 8061
```

```
lfq_qf[['peptides_filtered_missing']] <- QFeatures::filterNA(
  lfq_qf[['peptides_norm_hist']], 4/6)

biomasslmb:::message_parse(rowData(lfq_qf[['peptides_filtered_missing']]),
  'Leading.razor.protein',
  "Removing peptides with more than 4/6 missing values")
```

```
## 25263 features found from 3684 master proteins => Removing peptides with more than 4/6 missing values
```

7 Inspecting the number of peptides and proteins through the processing steps

Now that we have performed all the filtering steps, it's helpful to visualise how many peptides/proteins were retained at each level of the processing.

7.1 Samples per peptide

We'll start by inspecting the number of peptides in each experiment. We set the row variables to be **Sequence** and **Modifications** so that we count the number of unique peptides.

```

rename_cols <- c('All peptides' = 'peptides_raw' ,
                 'Quantified, contaminants removed' = 'peptides_filtered',
                 'Max 4/6 missing values' = 'peptides_filtered_missing')

rowvars <- c('Sequence', 'Modifications')
rowDataNames(lfq_qf)[['peptides_raw']]

```

```

## [1] "Leading.razor.protein"      "Sequence"
## [3] "Modifications"            "Mass"
## [5] "Mass.Fractional.Part"     "Protein.Groups"
## [7] "Proteins"                  "Gene.Names.x"
## [9] "Protein.Names"            "Unique..Groups."
## [11] "Unique..Proteins."        "Acetyl..Protein.N.term."
## [13] "Oxidation..M."            "Missed.cleavages"
## [15] "Experiment.A"              "Experiment.B"
## [17] "Experiment.C"              "Experiment.D"
## [19] "Experiment.F"              "Experiment.G"
## [21] "Retention.time"           "Calibrated.retention.time"
## [23] "Charges"                   "PEP"
## [25] "MS.MS.scan.number"        "Raw.file"
## [27] "Score"                     "Delta.score"
## [29] "Reverse"                   "Potential.contaminant"
## [31] "Intensity"                 "id"
## [33] "Protein.group.IDs"         "Peptide.ID"
## [35] "Evidence.IDs"              "MS.MS.IDs"
## [37] "Best.MS.MS"                "Oxidation..M..site.IDs"
## [39] "MS.MS.Count"               "Taxonomy.IDs"
## [41] "Taxonomy.names"           "Mass.deficit"
## [43] "Entry.Name"                "Reviewed"
## [45] "Protein.names"             "Gene.Names.y"
## [47] "Organism"                  "Length"
## [49] "Gene.Names.First"          "Q.value"

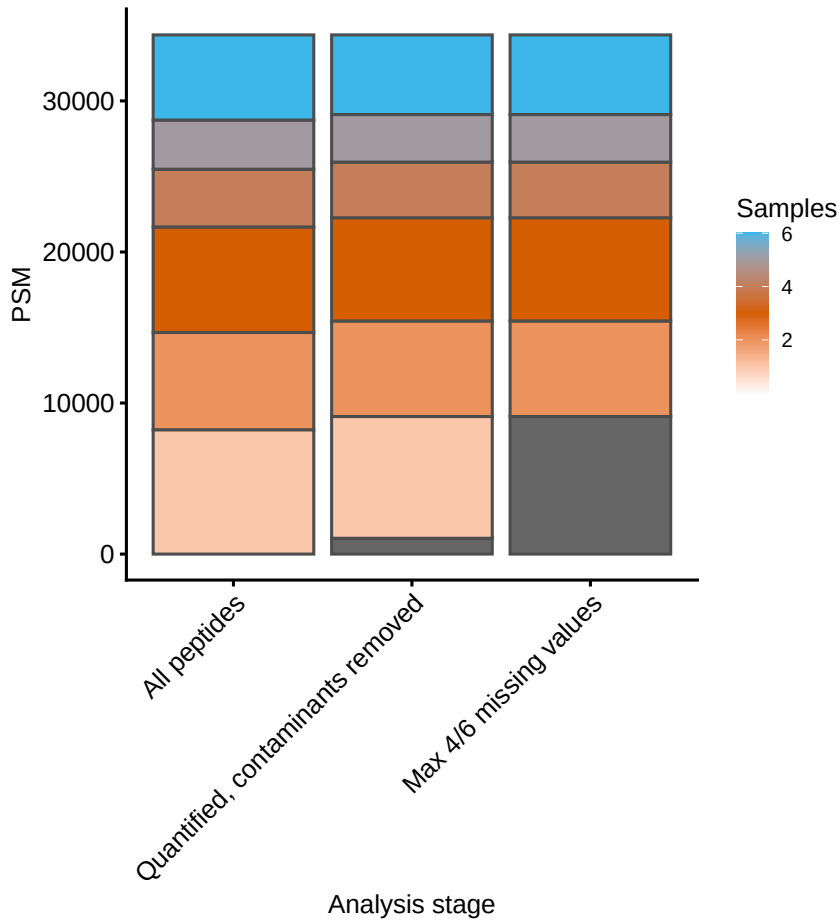
```

```

samples_present <- get_samples_present(
  lfq_qf[, , unname(rename_cols)], rowvars, rename_cols)

plot_samples_present(samples_present, rowvars, breaks=seq(2,10,2)) + ylab('PSM')

```

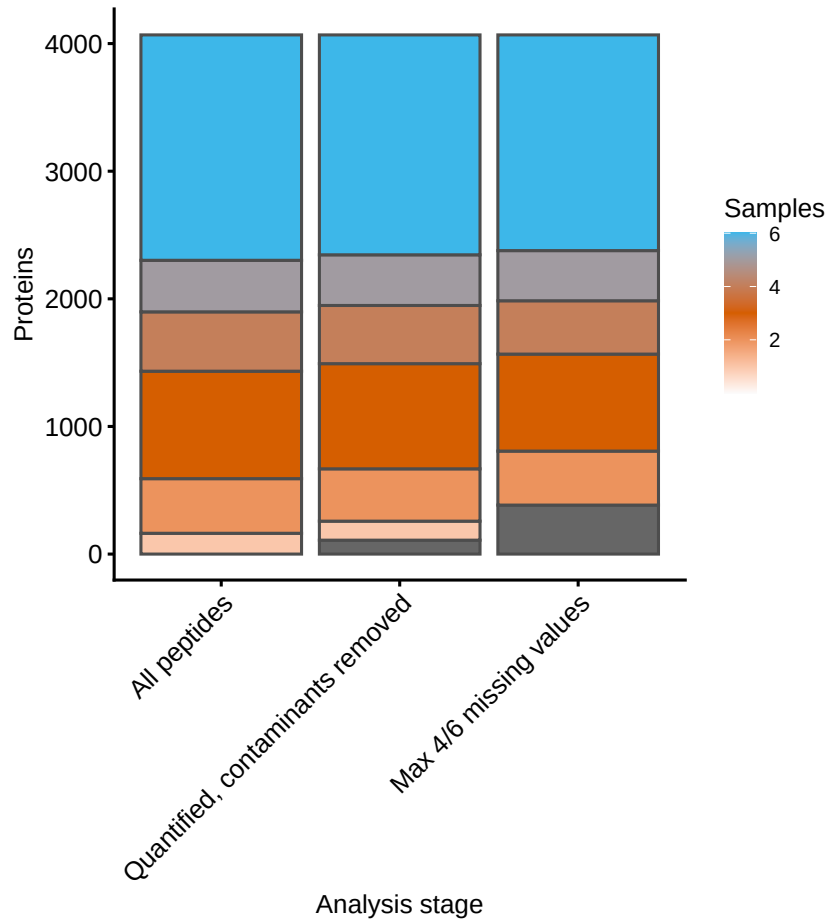


7.1.1 Samples per Protein

Next, we'll use the same functions to inspect the number of proteins at each level of processing. We need to set the row variables to be just the `Leading.razor.protein` column.

```
rowvars_prot <- c('Leading.razor.protein')

samples_present <- get_samples_present(lfq_qf, rowvars_prot, rename_cols)
plot_samples_present(samples_present, rowvars_prot, breaks=seq(2,10,2))
```



8 Save outputs

Save the full `QFeatures` object (all processing stages) for use in notebook 2.

```
output_dir <- here('notebooks/output')
saveRDS(lfq_qf, here(output_dir, 'HALO_lfq_qf.rds'))
```

9 Session info

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.6.2
##
## Matrix products: default
## BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK v
```

```

##
## locale:
## [1] en_GB.UTF-8/en_GB.UTF-8/en_GB.UTF-8/C/en_GB.UTF-8/en_GB.UTF-8
##
## time zone: Europe/London
## tzcode source: internal
##
## attached base packages:
## [1] stats4      stats      graphics  grDevices  utils      datasets  methods
## [8] base
##
## other attached packages:
## [1] xml2_1.6.0           QFeatures_1.20.0
## [3] MultiAssayExperiment_1.36.2 SummarizedExperiment_1.40.0
## [5] Biobase_2.70.0       GenomicRanges_1.62.1
## [7] Seqinfo_1.0.0        IRanges_2.44.0
## [9] S4Vectors_0.48.1     BiocGenerics_0.56.0
## [11] generics_0.1.4       MatrixGenerics_1.22.0
## [13] matrixStats_1.5.0    here_1.0.2
## [15] dplyr_1.2.1          tidyr_1.3.2
## [17] ggplot2_4.0.3        biomasslmb_0.1.0
##
## loaded via a namespace (and not attached):
## [1] tidyselect_1.2.1      farver_2.1.2          blob_1.3.0
## [4] Biostrings_2.78.0     S7_0.2.2              fastmap_1.2.0
## [7] lazyeval_0.2.3        XML_3.99-0.24         digest_0.6.39
## [10] lifecycle_1.0.5       cluster_2.1.8.3       ProtGenerics_1.42.0
## [13] survival_3.8-12       KEGGREST_1.50.0       RSQLite_3.53.3
## [16] magrittr_2.0.5        genefilter_1.92.0     compiler_4.5.2
## [19] rlang_1.3.0           tools_4.5.2           igraph_2.3.3
## [22] yaml_2.3.12           corrrplot_0.95        knitr_1.52
## [25] labeling_0.4.3        S4Arrays_1.10.1       bit_4.6.0
## [28] DelayedArray_0.36.1   plyr_1.8.9            RColorBrewer_1.1-3
## [31] abind_1.4-8           withr_3.0.3           purrr_1.2.2
## [34] grid_4.5.2            xtable_1.8-8          scales_1.4.0
## [37] MASS_7.3-65           cli_3.6.6             UpSetR_1.4.1
## [40] rmarkdown_2.32        crayon_1.5.3          otel_0.2.0
## [43] robustbase_0.99-7     httr_1.4.9            reshape2_1.4.5
## [46] BiocBaseUtils_1.12.0  DBI_1.3.0             cachem_1.1.0
## [49] stringr_1.6.0         splines_4.5.2         AnnotationDbi_1.72.0
## [52] AnnotationFilter_1.34.0 XVector_0.50.0        vctrs_0.7.3
## [55] Matrix_1.7-4          naniar_1.1.0          bit64_4.8.6
## [58] visdat_0.6.0          clue_0.3-68           annotate_1.88.0
## [61] glue_1.8.1            DEoptimR_1.2-1        uniprotREST_1.0.0
## [64] stringi_1.8.9         gtable_0.3.6          tibble_3.3.1
## [67] pillar_1.11.1         htmltools_0.5.9       R6_2.6.1
## [70] rprojroot_2.1.1       evaluate_1.0.5        lattice_0.23-1
## [73] backports_1.5.1       png_0.1-9             memoise_2.0.1
## [76] Rcpp_1.1.2            gridExtra_2.3.1       checkmate_2.3.4
## [79] SparseArray_1.10.10   xfun_0.61             MsCoreUtils_1.22.1
## [82] pkgconfig_2.0.3

```